metro_ridership_app — architecture

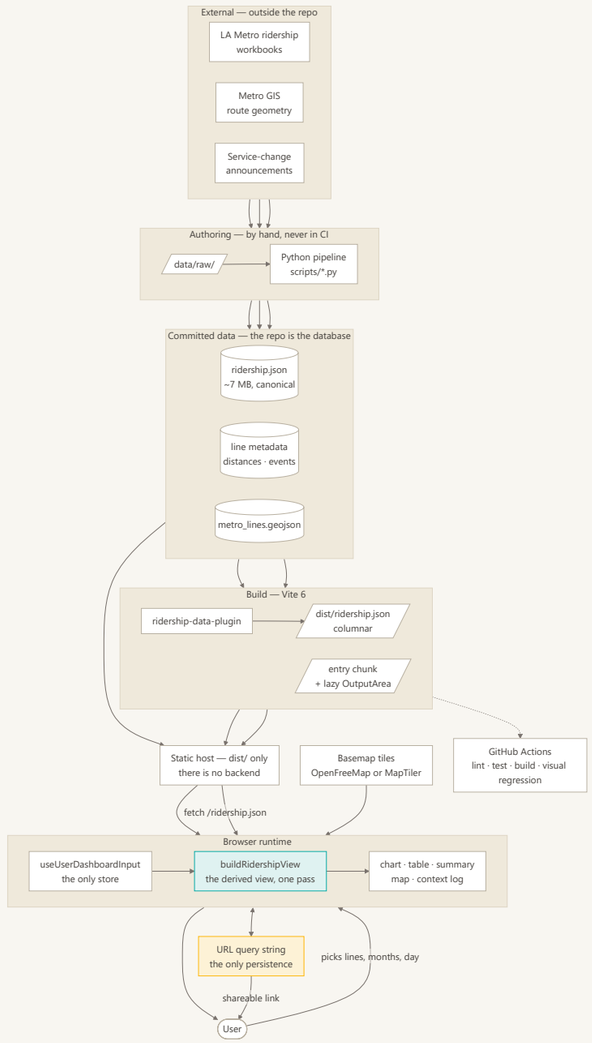
A whole-system view plus one diagram per subsystem, generated from the mermaid sources in `docs/architecture/mermaid/`. Drawn in the dashboard's own palette, with accents taken from the Metro line colours in `definedLines`: **green** a public surface, **amber** accepted but not yet wired in, **red** a cycle or a duplication risk. The diagram card keeps that palette in a dark theme too.

CONTENTS

01 The whole system

The one view that contains everything else. Three stages, and the boundaries between them are what matter: a **Python pipeline** that runs by hand and writes JSON into the repo, a **Vite build** that re-encodes that JSON into a wire format, and a **browser** that derives everything on screen from one set of user choices.

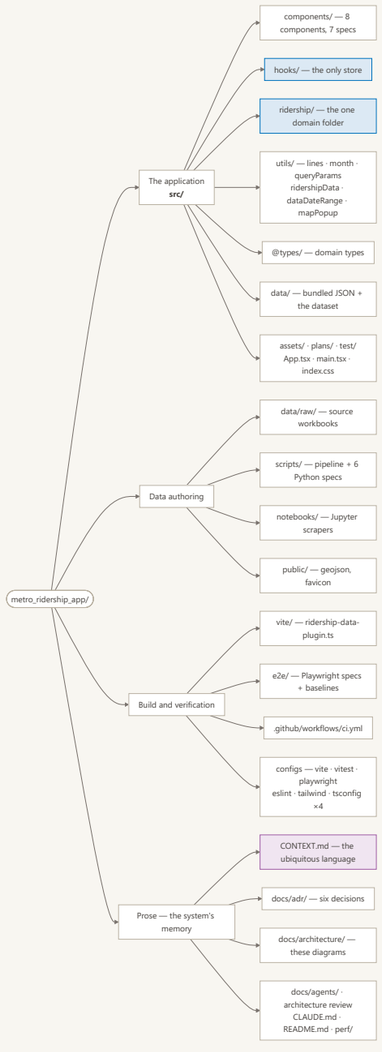
There is no backend. The repository *is* the database, `dist/` is the whole deployment, and the URL query string is the only thing that survives a reload. Every remaining diagram drills into one box here.



01-whole-system.mmd

02 Repository map

What lives where. Two things are worth noticing. `src/` is flat — `components/`, `hooks/`, `utils/`, `data/`, `@types/` — with exactly one domain folder, `src/ridership/`, and ADR-0003 says that is deliberate rather than the first step of a reorganisation. And the repo carries an unusual amount of prose: `CONTEXT.md`, six ADRs, an architecture review, seven design plans. That is the system's memory; read it before changing behaviour that looks wrong.

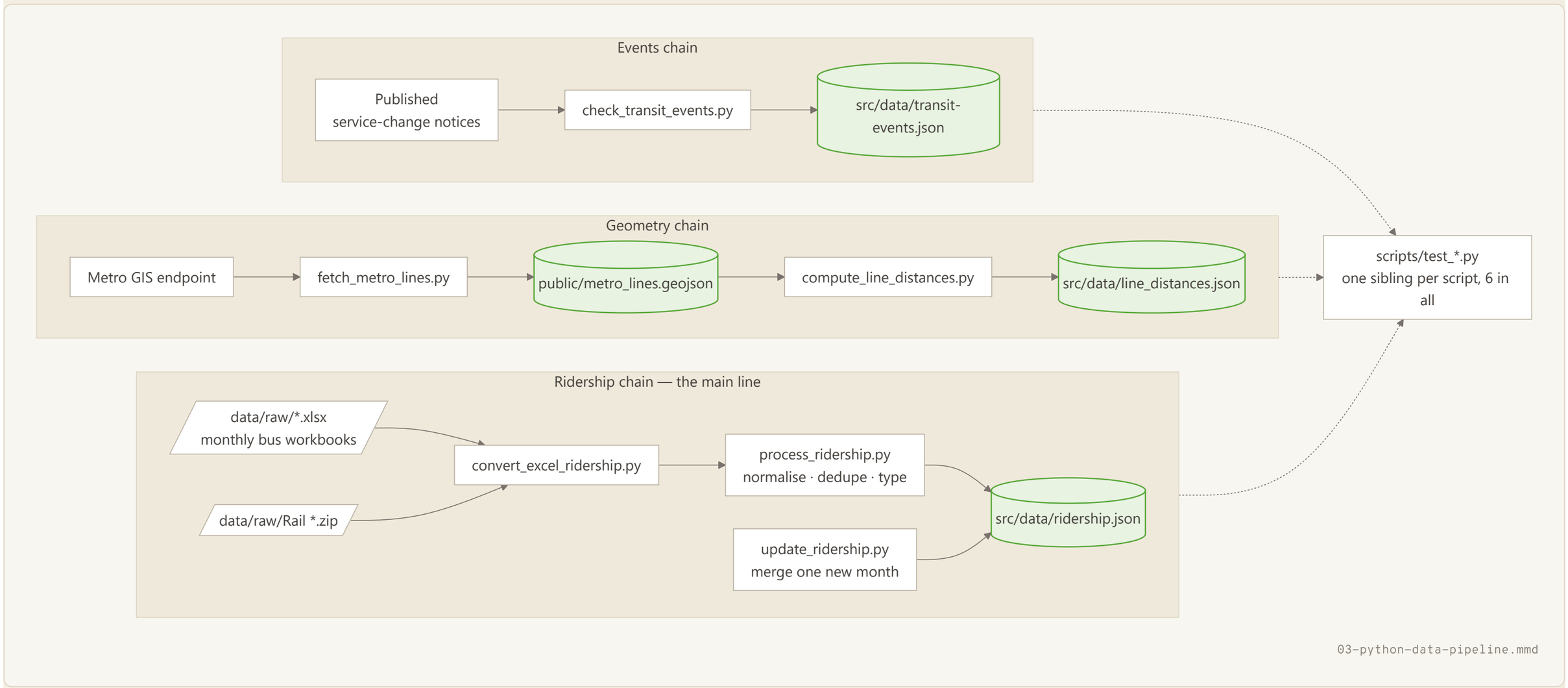


02-repository-map.mmd

03 The Python data pipeline

How data gets into the repo: three independent chains, left to right. Nothing here runs in CI — a human runs these scripts and commits the result, which is why `src/data/ridership.json` is a checked-in 7 MB file rather than a build artifact.

`update_ridership.py` is the incremental path: when Metro publishes a single new month, it merges that month into the canonical file instead of rebuilding it from every workbook. Every script has a `test_*.py` sibling — six in all.

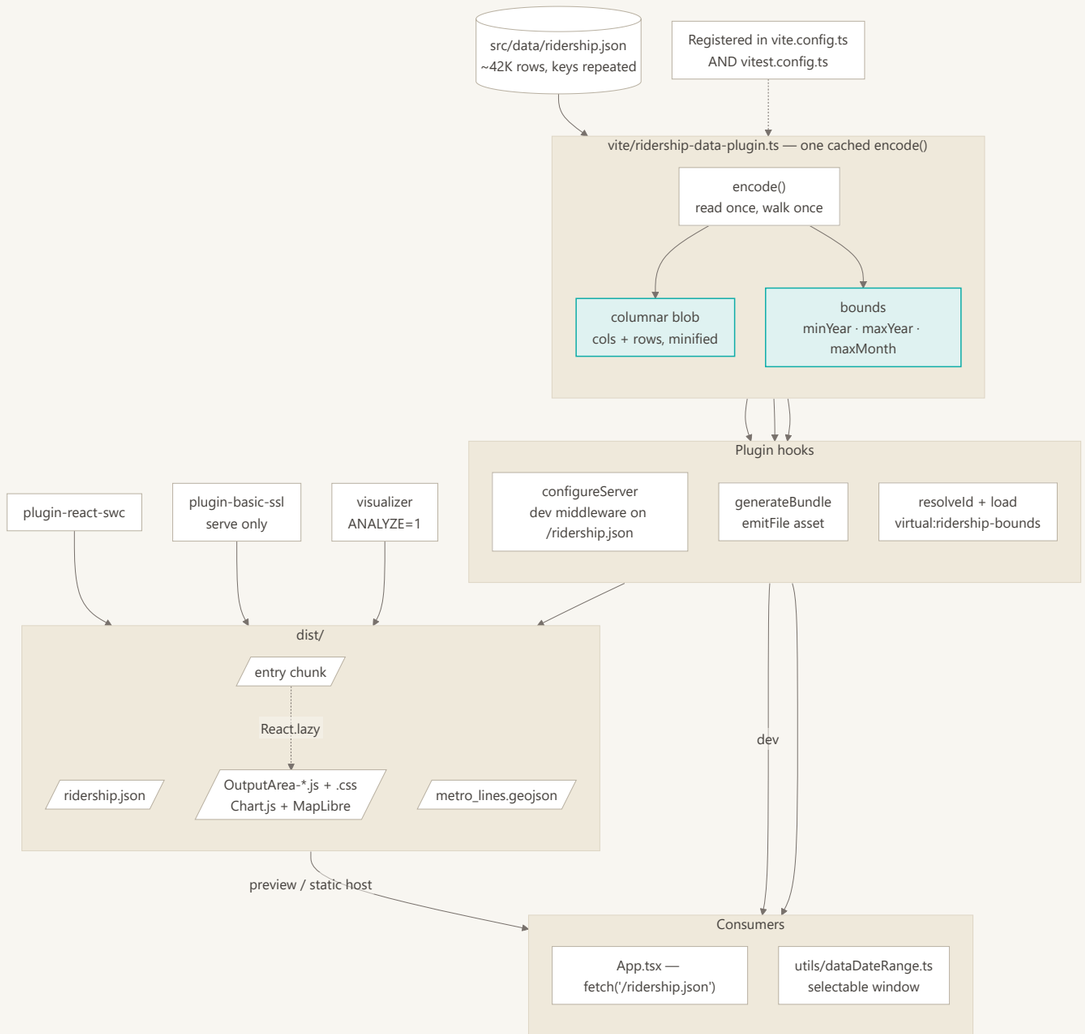


03-python-data-pipeline.mmd

04 The build pipeline

`vite/ridership-data-plugin.ts` is the interesting part. The canonical JSON repeats six field names on every one of ~42K rows and, imported normally, would inline about 6.6 MB of object literal into the entry chunk. The plugin reads it once and produces two things from that single cached pass: a minified columnar blob served at `/ridership.json`, and a `virtual:ridership-bounds` module carrying just the min/max year and latest month.

The blob reaches the app two different ways — dev middleware in `configureServer`, an emitted asset in `generateBundle` — so the runtime `fetch` is identical in both. The plugin is registered in `vitest.config.ts` as well, or the virtual module would not resolve under the test runner.

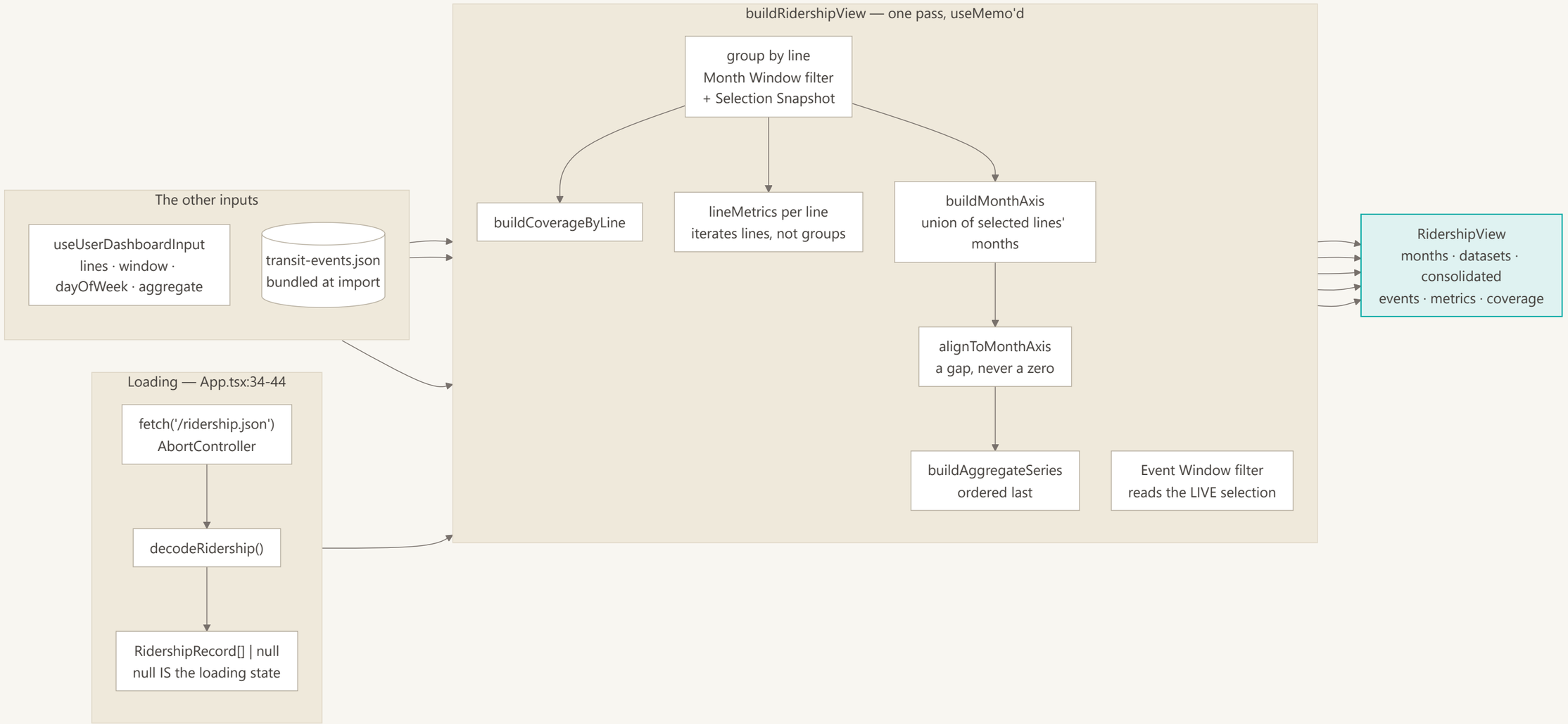


04-build-pipeline.mmd

05 Loading and deriving

The core architecture, first half. Records are fetched rather than bundled, decoded from the columnar blob, and handed with the user's choices to a single `buildRidershipView` call that produces the whole derived view in one pass.

`null` is the loading state, not an empty array — the distinction is what lets the app filter and show context-log events while the ridership data is still in flight. Note that the metrics loop iterates `lines`, not the consolidated groups: a record whose `line_name` has no metadata entry produces a group but no `Line`, and therefore no figures.



05-runtime-load-and-derive.mmd

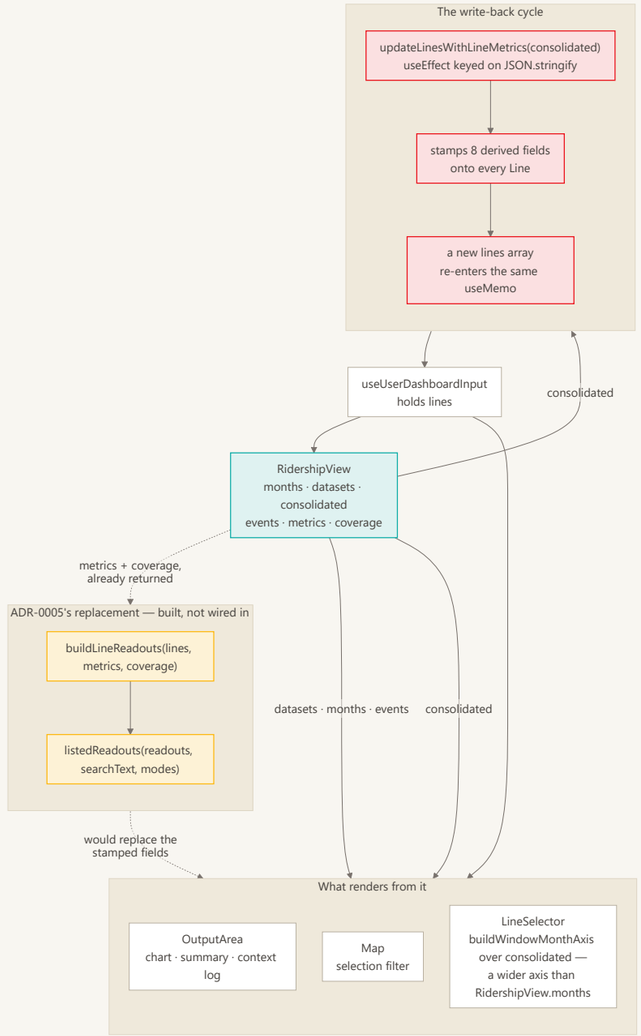
06 Consuming the view, and the write-back

The second half, and the one live design problem in the app.

`buildRidershipView` already returns `metrics` and `coverage` keyed by line id — everything a caller needs — yet `App.tsx:94` still calls `updateLinesWithLineMetrics`, which writes eight derived fields back onto every `Line`. That mints a new `lines` array, which re-enters the memo it came from; the `JSON.stringify` dependency keys exist to keep the loop from thrashing. It settles only because the second pass produces figures identical to the first.

ADR-0005 accepted removing it. `buildLineReadouts` and `listedReadouts` are the replacement — written, unit-tested, and imported by nothing that renders.

`LineSelector` reads `consolidated` directly and builds its own axis, because the table draws a sparkline for every *visible* line while the chart covers only the *selected* ones.

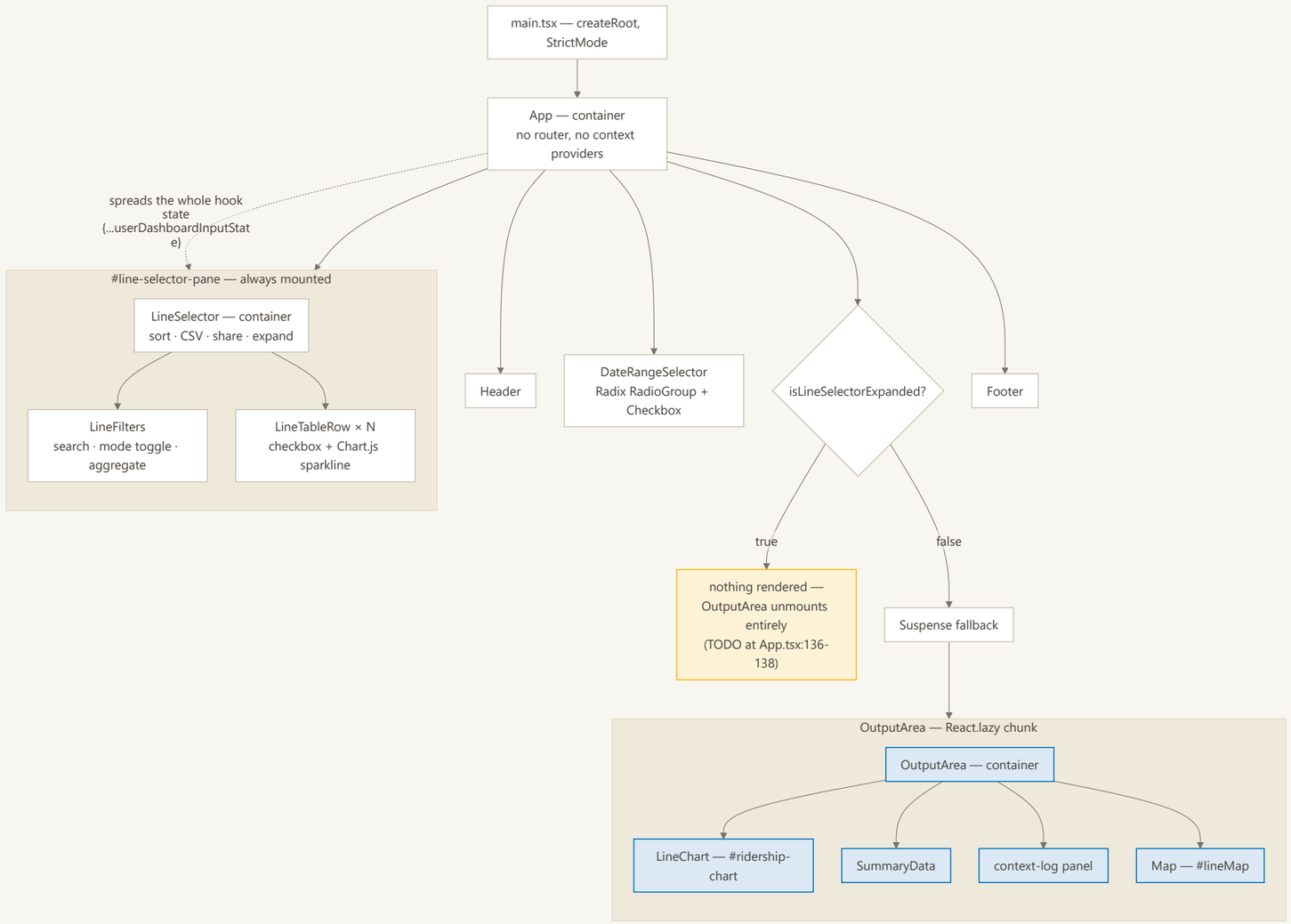


06-runtime-consume-and-writeback.mmd

07 Component tree

Eight components, no router, no context providers. `App` spreads the entire hook state into `LineSelector` with `{...userDashboardInputState}`, so that component's real interface is far wider than its props list suggests.

`OutputArea` is `React.lazy` on purpose: it pulls in Chart.js and MapLibre, and keeping them out of the entry chunk lets the header and line table paint first. Note the gate — expanding the line selector *unmounts* `OutputArea` rather than hiding it, so the chart and map rebuild from scratch on collapse. `App.tsx:136-138` flags this.

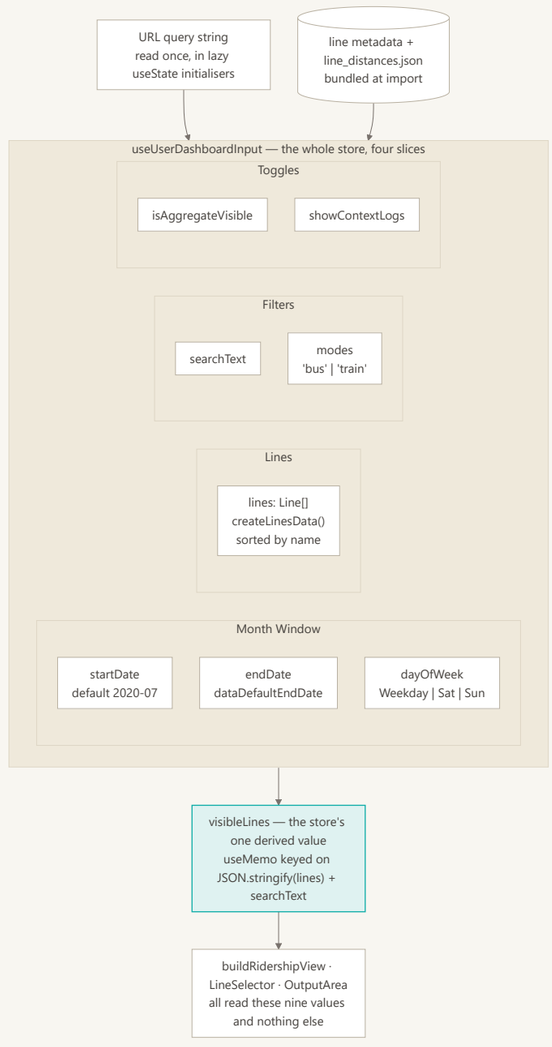


07-component-tree.mmd

08 State slices

All shared state lives in one custom hook: four slices, no Redux, no Zustand, no Context. Each slice is seeded once from the URL in a lazy `useState` initialiser, so a shared link reconstructs the view before the first render rather than after it.

`visibleLines` is the only derived value in the store, and its memo key is `JSON.stringify(lines)` rather than `lines` — see the next diagram for why.



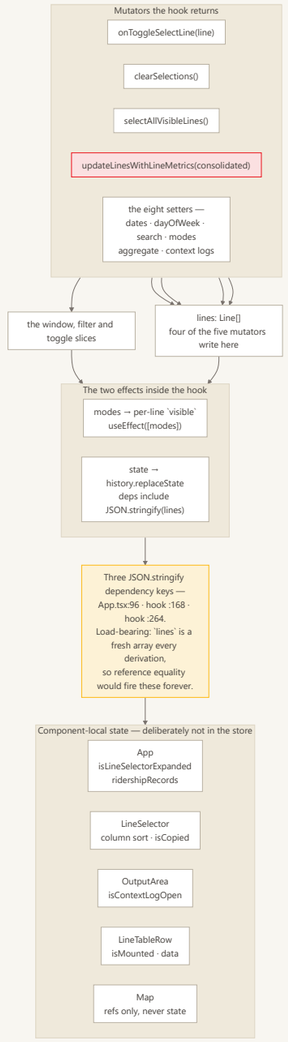
08-state-slices.mmd

09 Mutators, effects, and local state

Everything that writes. Four of the five mutators touch `lines`, which is why that one array is the hinge the whole store turns on.

The `JSON.stringify` dependency keys at `App.tsx:96`, `useUserDashboardInput.ts:168` and `:264` are load-bearing, not sloppiness: `lines` is a fresh array on every derivation, so reference equality would fire these effects forever. `CLAUDE.md` asks you not to "fix" them. The real fix is removing the write-back that mints the array (ADR-0005), not changing the keys.

What is *not* in the store matters too. Expansion, the fetched records, sort state, the context-log disclosure and every MapLibre handle stay local, so none of them participate in the derivation above.

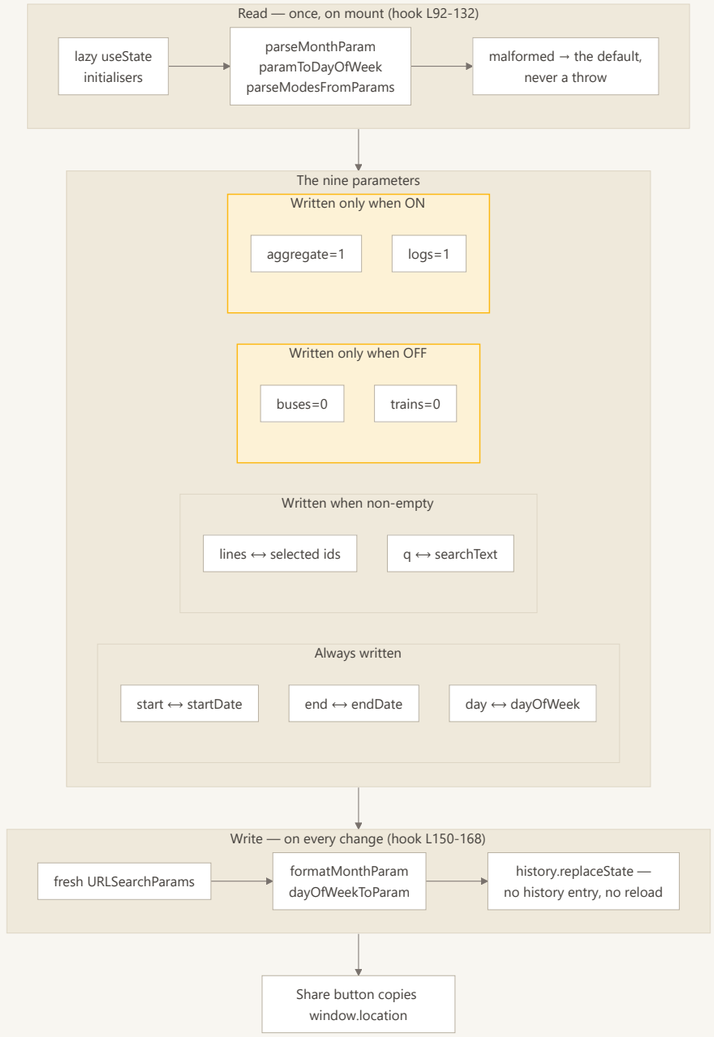


09-state-mutators-and-effects.mmd

10 The URL contract

Nine parameters, read once into lazy `useState` initialisers and written back with `history.replaceState` on every change. No router, no `localStorage`, no server — this is the app's entire persistence layer, and the reason every view is a shareable link.

The contract is asymmetric by design: `buses`/`trains` are written only when *off*, `aggregate`/`logs` only when *on*, which keeps the common URL short. Malformed values fall back to defaults rather than throwing. Nine ad-hoc reads and one hand-built writer are what candidate 5 of the architecture review would replace with an explicit parsed contract; it is unscheduled.

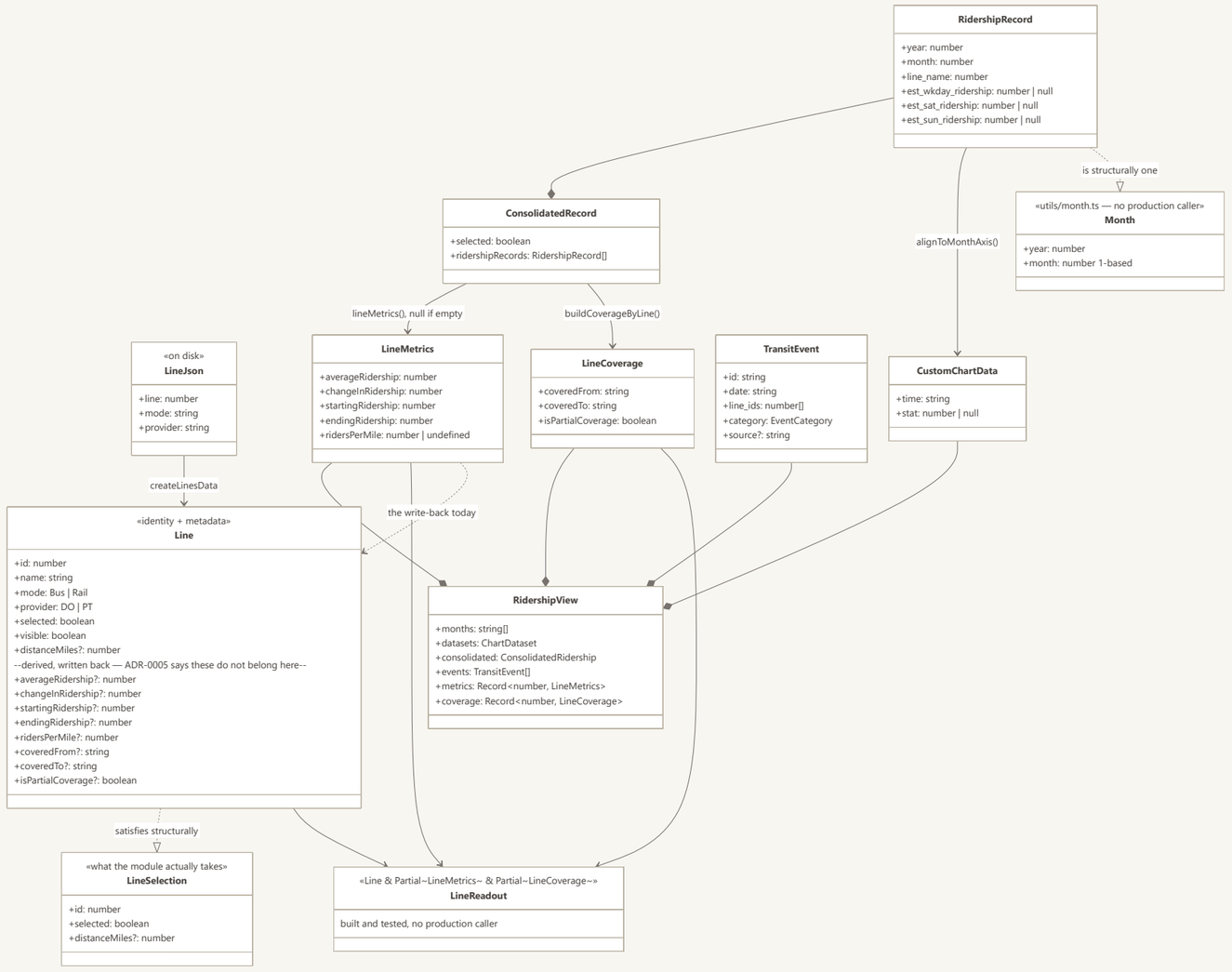


10-url-contract.mmd

11 Domain type model

The types and how they relate. Read `Line` from the top down: identity and metadata first, then a block of optional derived figures that ADR-0005 says do not belong there. `LineSelection` is the same information minus that block — it is what `buildRidershipView` actually accepts, and `Line` satisfies it structurally, which is what keeps derived figures from being handed back into the module that produced them.

`LineReadout` is the intended destination: `Line & Partial<LineMetrics> & Partial<LineCoverage>`, derived per window and thrown away. `Month` is ADR-0006's replacement for the seven encodings a month currently has. Both exist; neither is wired in.

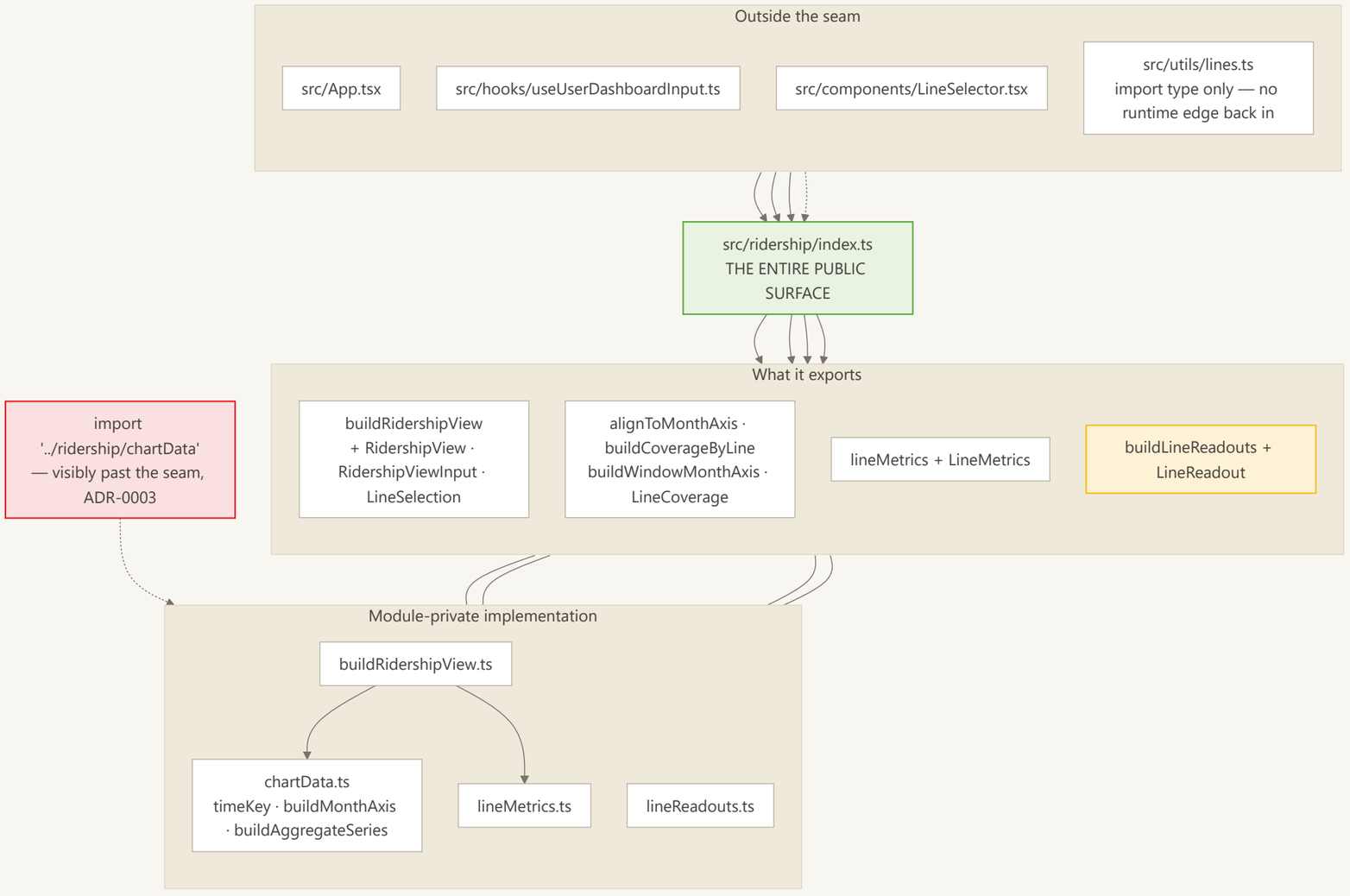


11-domain-type-model.mmd

12 The `src/ridership/` seam

`index.ts` is the module's entire public surface. Everything else in the folder is implementation, so an import of `../ridership/chartData` from outside is *visibly* reaching past a seam — which is the whole point of the folder existing (ADR-0003). A flat `src/utils/ridershipView.ts` could only have asked for that in a comment.

The month-axis and coverage exports are a deliberate second entry point rather than a leak. `buildRidershipView` derives the **chart**, over the **selected** lines only; the line table draws a sparkline for every **visible** line and needs the wider union across all of `consolidated`.



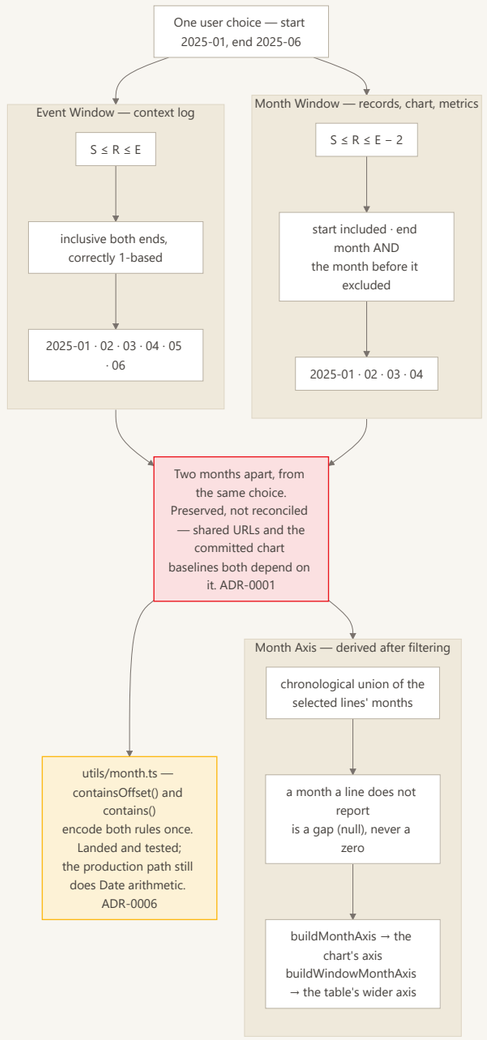
12-ridership-module-seam.mmd

13 Month Window, Event Window, Month Axis

The single most surprising thing in the codebase. One user choice produces two windows that disagree by two months: records use `S ≤ R ≤ E − 2` — the start month is in, the end month **and the month before it** are out — while the context log uses an ordinary inclusive range.

This reads like an off-by-one and is not. It is long-standing behaviour, users have shared URLs against it, and `e2e/chart-content.spec.ts` renders windows through it into committed PNG baselines, so normalising it would change what every existing link shows. ADR-0001 accepts it.

The Month Axis is derived after filtering: one shared axis for every series, because Chart.js appends any label missing from `labels` to the end and a per-series axis scrambles the rest.

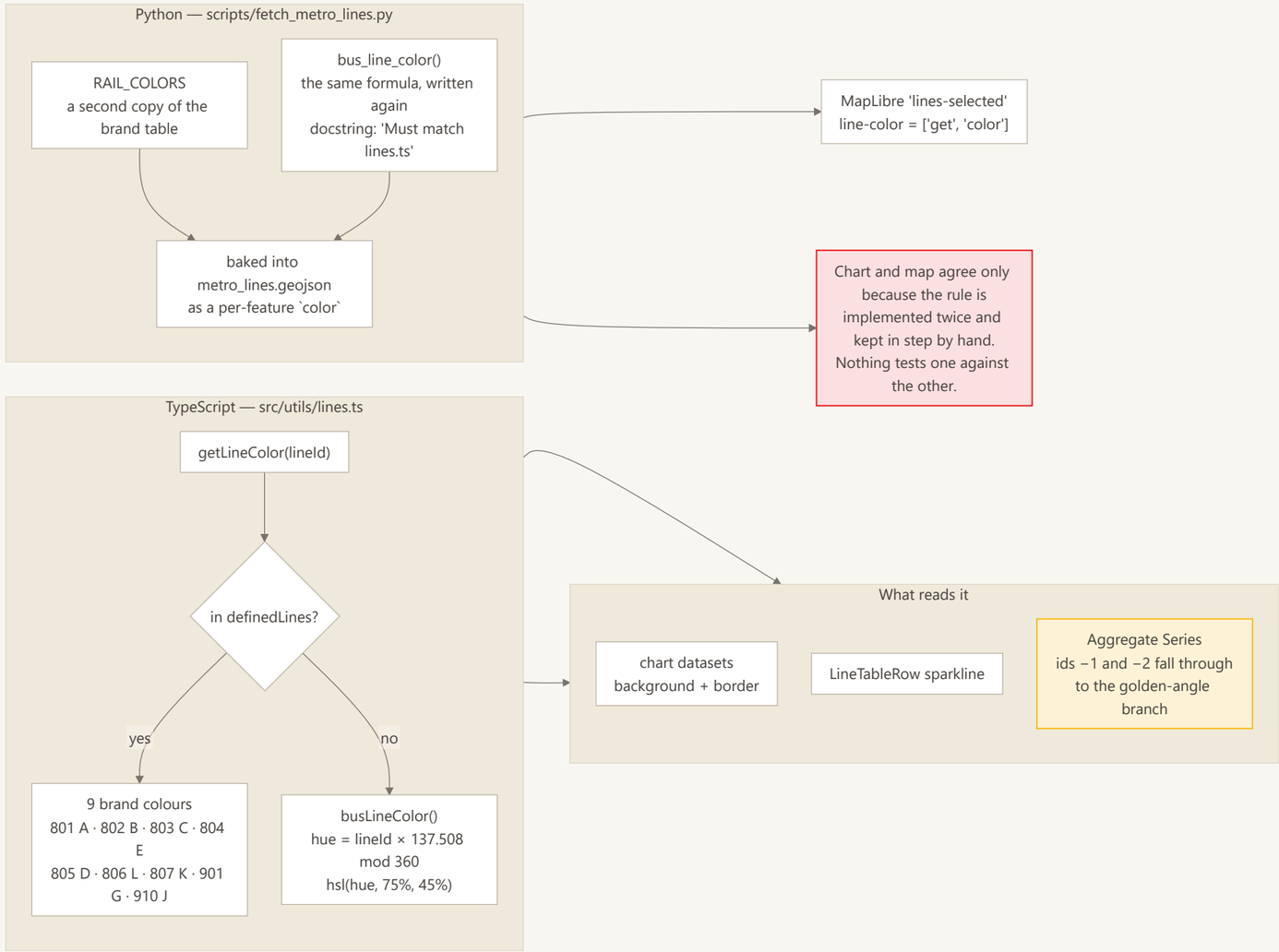


13-month-windows.mmd

14 Line colour resolution

Nine rail and BRT lines carry hardcoded brand colours; every other line gets a deterministic golden-angle hue, so a bus line looks the same on every render without anything being stored. These are the colours this document is drawn in.

The honest part of the diagram is the right-hand branch. The map does **not** call `getLineColor` — MapLibre paints from a `color` property baked into `metro_lines.geojson` by `scripts/fetch_metro_lines.py`, which reimplements the same formula and the same brand table in Python with a docstring reading "Must match lines.ts". Changing one desynchronises the map until the geojson is regenerated, and no test would catch it.

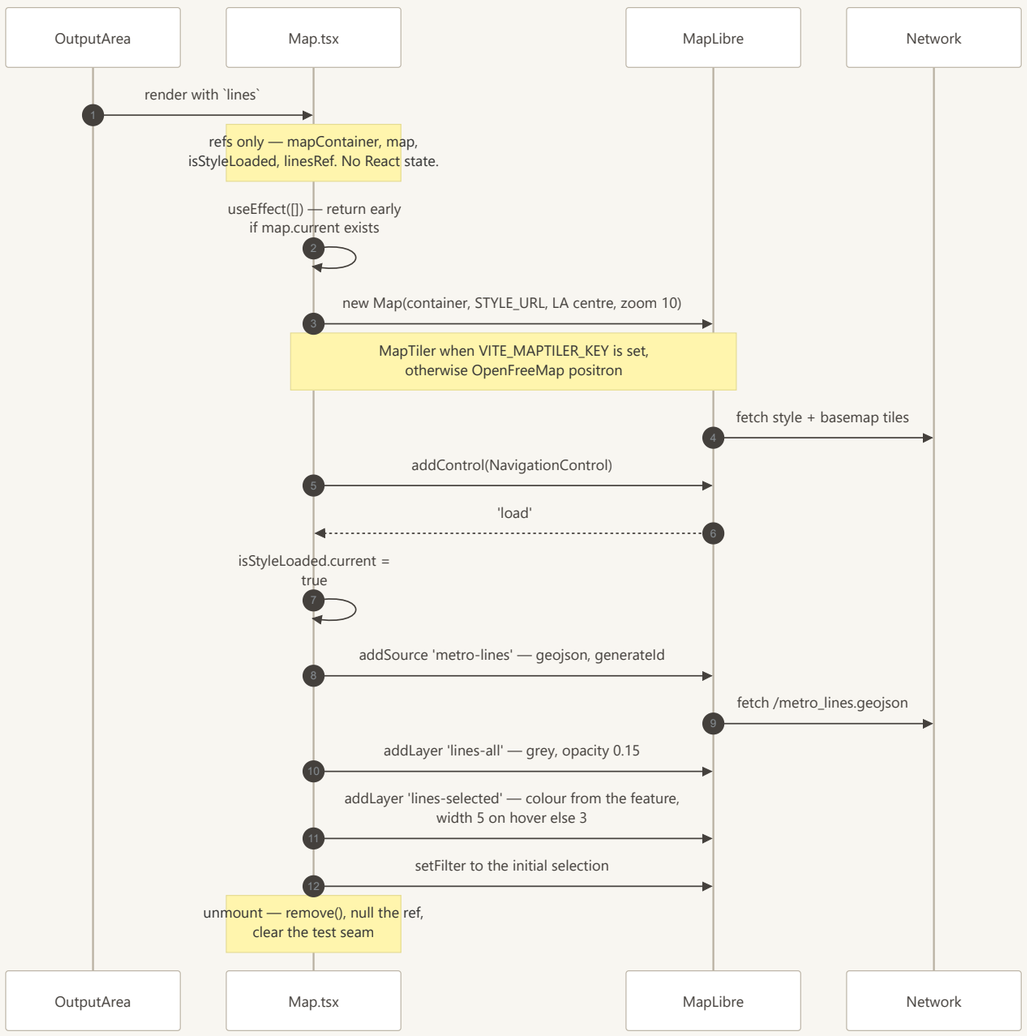


14-line-color-resolution.mmd

15 Map lifecycle

The one imperative corner of an otherwise declarative app. MapLibre owns its own canvas, so `Map.tsx` holds everything in refs and the component never re-renders on map state: one `useEffect([])` builds the map, adds the two layers once the style has loaded, and tears it all down on unmount.

Layer order is load-bearing — `lines-all` paints every route dimmed underneath, `lines-selected` paints the chosen ones on top in brand colour, so selection reads as emphasis rather than as the only thing on the map.

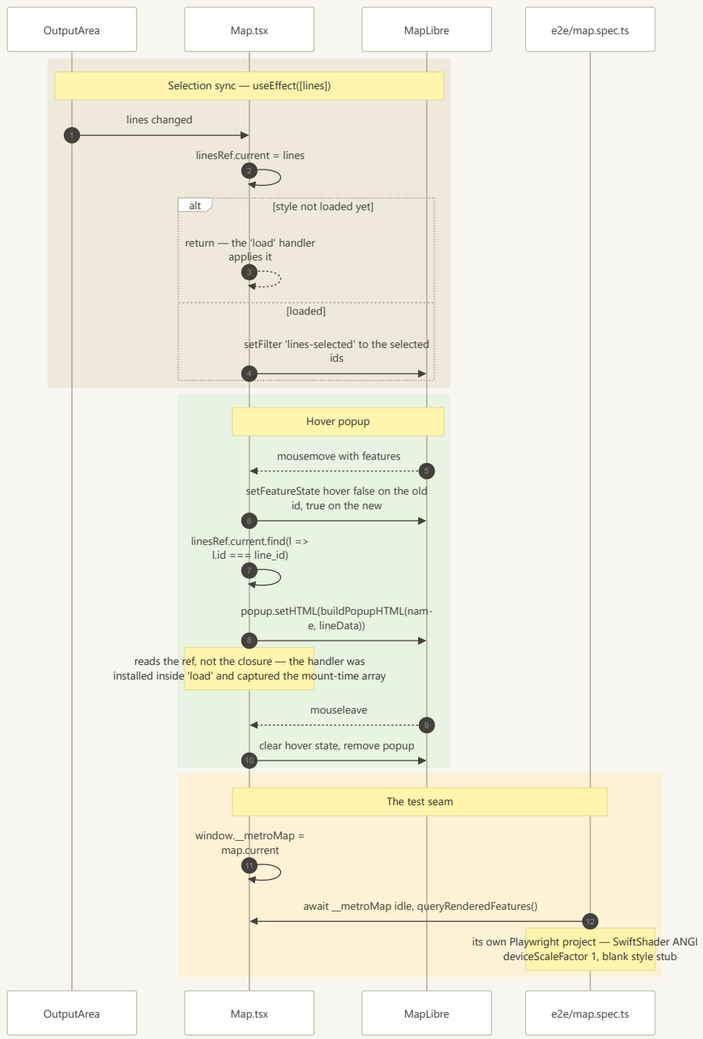


15-map-lifecycle.mmd

16 Map interaction and the test seam

What happens after the map exists. Selection changes only update a layer filter; the map is never rebuilt.

Two details are load-bearing. The hover handler reads `linesRef.current` rather than the `lines` closure, because it is installed inside the `load` callback and would otherwise capture the mount-time array forever. And `window.__metroMap` exists purely so `e2e/map.spec.ts` has something to await — a WebGL canvas gives the DOM no signal that it has finished drawing. Nothing in the app reads it; don't delete it.

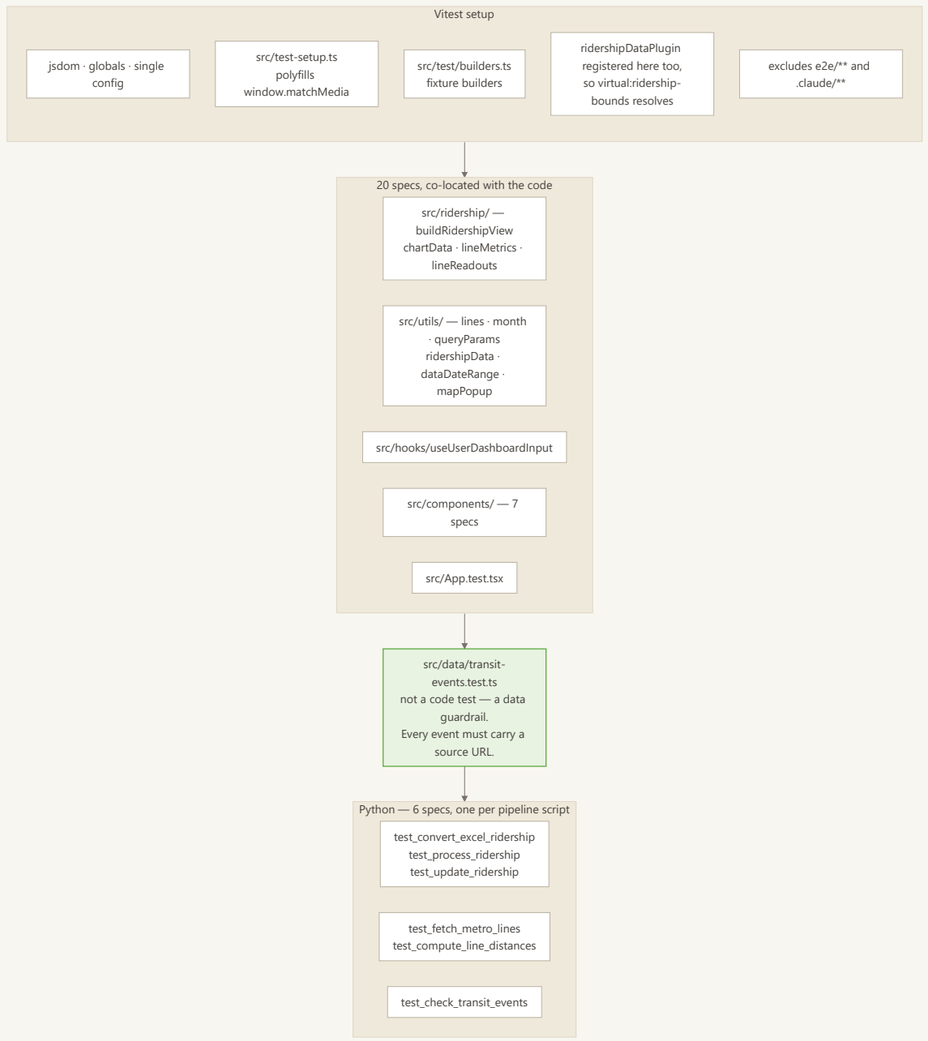


16-map-interaction.mmd

17 Unit and Python suites

Vitest runs 20 co-located specs in jsdom. One of them is not a code test at all: `src/data/transit-events.test.ts` refuses to let an event ship without a source URL — the type makes `source` optional so fixtures stay cheap, and the guardrail closes the gap.

The Python side mirrors the pipeline exactly, one spec per script.

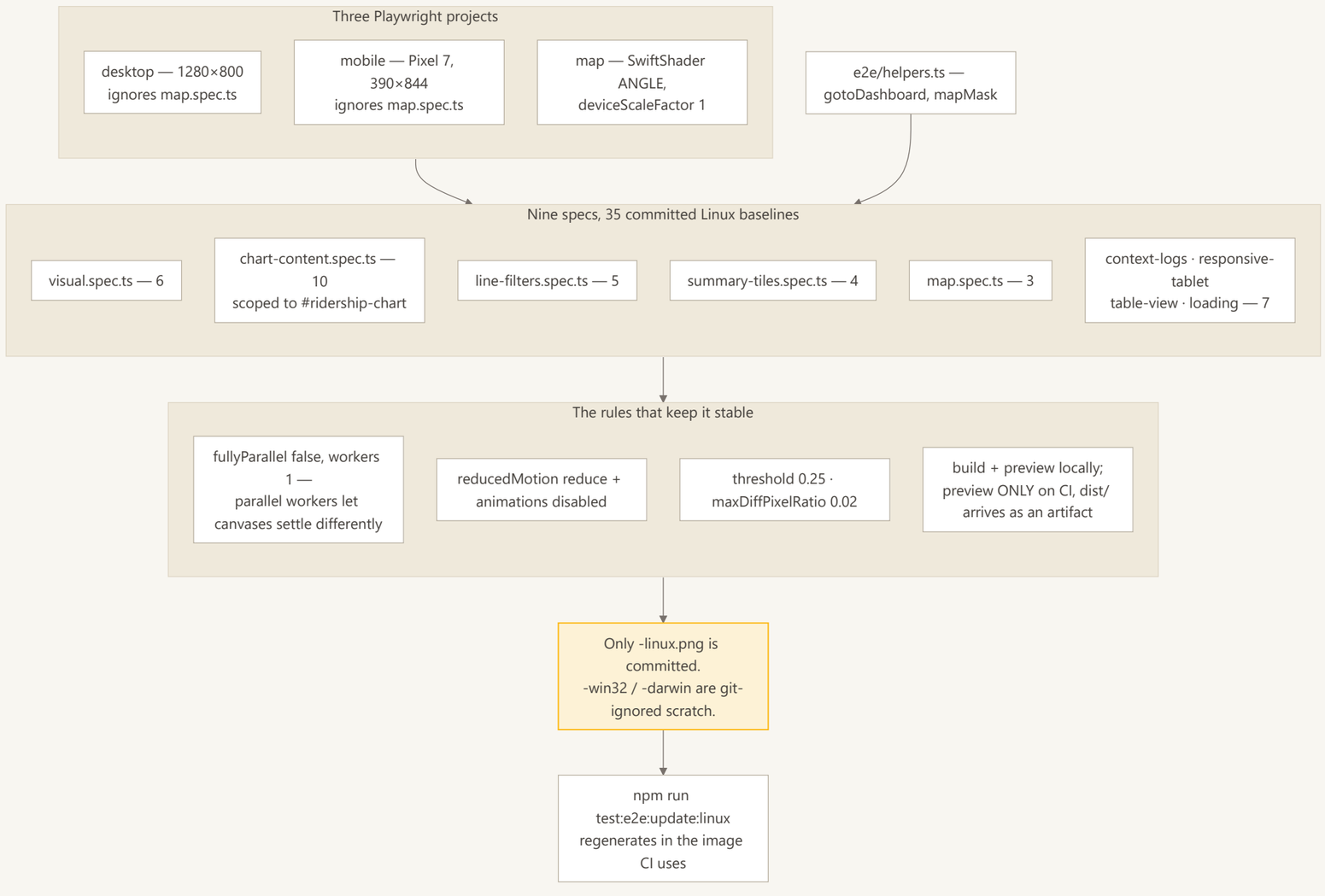


17-test-unit-and-python.mmd

18 Visual regression

Nine specs across three projects. The map gets its own because it renders identical geometry at any viewport, so running it twice would only double the flake surface.

Only `-linux.png` baselines are committed; Windows and macOS shots are git-ignored per-developer scratch. A UI change that moves pixels therefore needs exactly one command, `npm run test:e2e:update:linux`, which regenerates inside the same Docker image CI uses — the tag resolved from the same `package-lock.json` the workflow reads.

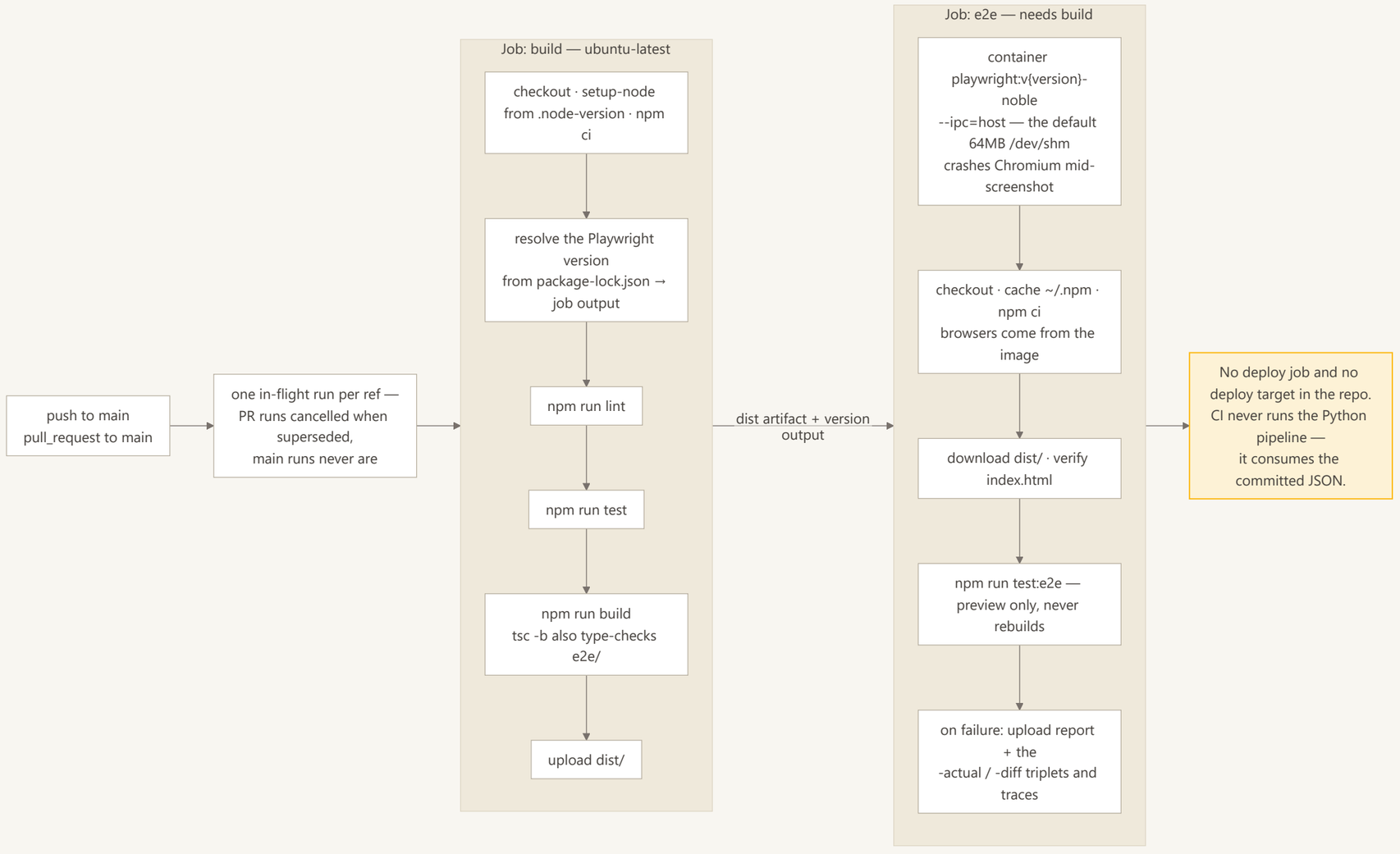


18-test-visual-regression.mmd

19 CI pipeline

Two jobs. `build` lints, unit-tests, builds and uploads `dist/`; `e2e` downloads that artifact and runs the visual suite against `vite preview`, so the app is built exactly once per run.

The Playwright container tag is derived from `package-lock.json` by `jq` and passed between jobs as an output, so the browser build that produced the committed baselines can never drift from the installed client.

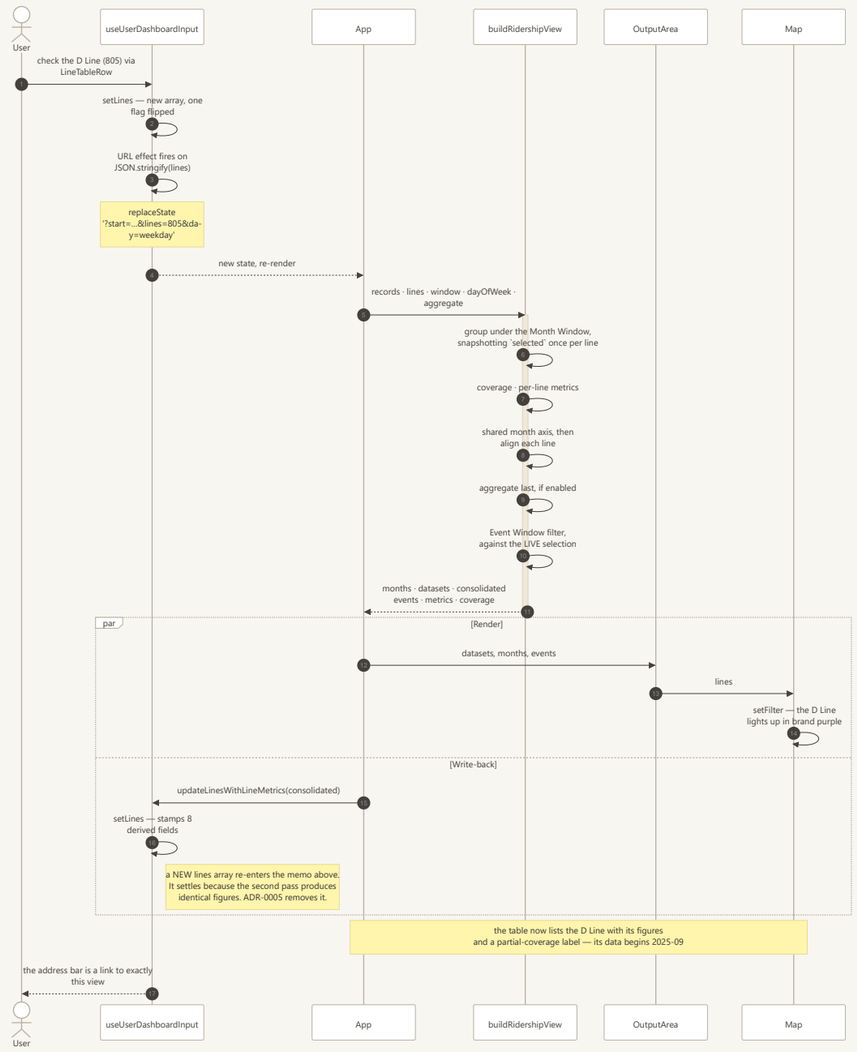


19-ci-pipeline.mmd

20 Selecting a line, end to end

One click traced through every layer, to show how the pieces above compose. The user checks a box; the hook mints a new `lines` array; the URL is rewritten; `buildRidershipView` re-derives the entire view in one pass; chart, table, summary and map all update from that one result.

The `par` block is the write-back cycle seen live: rendering and re-deriving happen alongside each other, and the loop settles only because the second pass produces figures identical to the first.

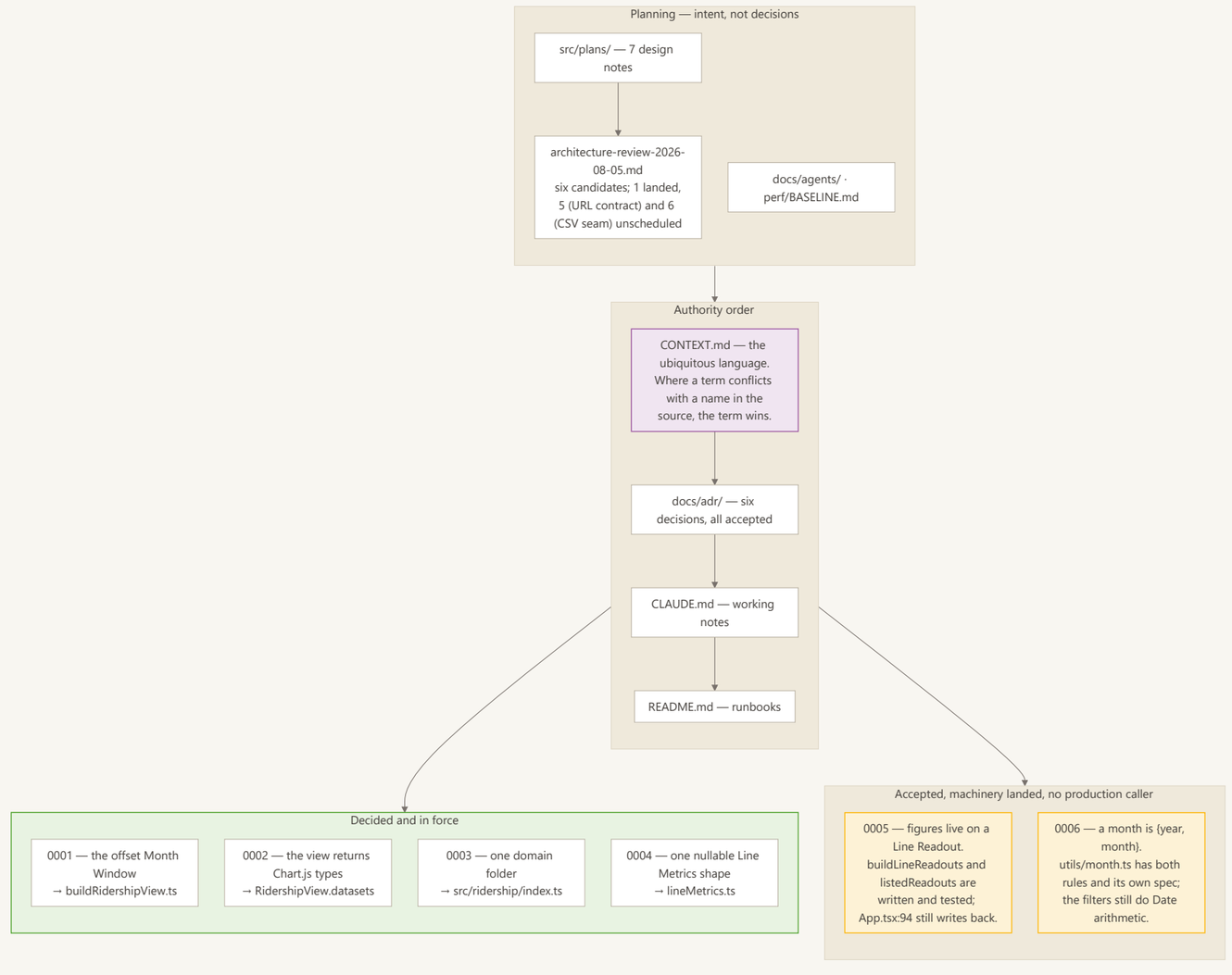


20-interaction-sequence.mmd

21 Documentation and decision map

Which document governs what, and where the code has not caught up. `CONTEXT.md` outranks the source by its own rule — where a term there conflicts with a name in the code, the code is what's out of date.

All six ADRs are accepted, but two are only half-landed: 0005's `buildLineReadouts` and 0006's `month.ts` both exist with full test coverage and no production caller. That gap is the most actionable thing in this whole set.



21-docs-adr-map.mmd

Generated by `scripts/build_architecture_docs.mjs` from `mermaid/` + `captions.md`. Do not edit this file directly.